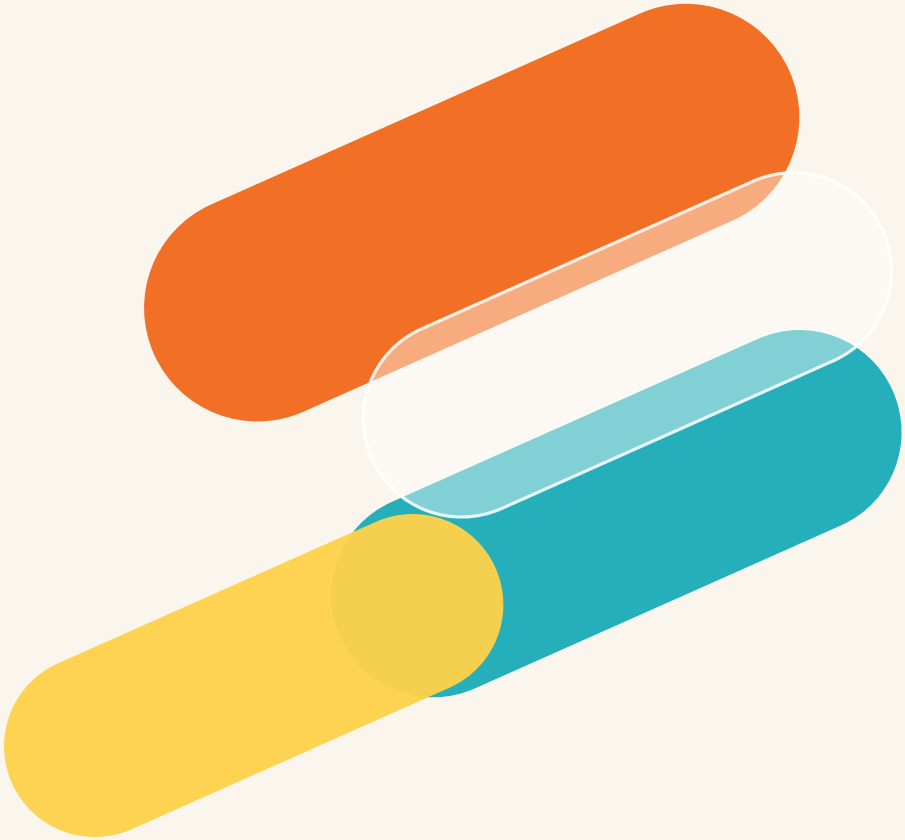


Vendora

A multi-vendor marketplace frontend

Built with React and TypeScript. Frontend only, ready for a real backend.

AUTHOR	Mohammad Talha Javaid

VERSION	1.0, October 2026
TIMELINE	Three build days and one QA day
APPROACH	Mock API with browser persistence; backend-ready service layer
STACK	React, TypeScript, Vite, Tailwind, React Router, TanStack Query, Zustand

Prepared for mentor review.

01

Project Overview

Vendora is an Amazon-style marketplace where customers browse one global catalog containing two kinds of listings: products sold directly by the platform (shown as **Sold by Vendora**) and products listed by independent vendors who control their own listings after being approved by the platform. The platform earns a commission on every vendor sale.

The application supports physical and digital products, six user roles, a multi-vendor cart, a simulated checkout, order lifecycle tracking, vendor and admin dashboards, dispute handling, and a support and delivery workflow. Everything runs in the browser against a mock API layer so the project can be demonstrated, deployed and tested without a backend.

The goal is not only a good-looking marketplace. The goal is to demonstrate a clean, scalable React architecture with correct state placement, structured asynchronous data handling, controlled re-rendering, and clear technical decisions.

1.1 What the project should demonstrate

- React fundamentals, component architecture and composition
- Server state versus client state separation
- API integration patterns, loading / error / empty / refetching states
- Race-condition handling, request cancellation and debounced search
- URL as state for search, filters, sorting and pagination
- Role-based access control across six roles
- Complex nested and relational state (carts, split orders, commissions)
- Form handling with validation, dynamic field arrays and multi-step flows
- Performance: rendering control, code splitting, lazy loading, image strategy
- Responsive, modern UI with advanced visual design and smooth motion
- Professional delivery: Git history, README, documentation, deployment, QA

02

Scope and Priorities

2.1 In scope

- Customer storefront: discovery, search, product pages, reviews, wishlist, comparison, recommendations
- Multi-vendor cart, checkout (simulated card and Cash on Delivery), orders, tracking, returns, disputes
- Vendor application and approval, vendor dashboard, product and order management, earnings
- Admin panel: vendor approval, product moderation, user management, category management, commission settings, reports, dispute handling
- Support agent workspace and delivery rider workspace
- Simulated authentication, simulated images, simulated payments

- Mock API with persistence, seeded realistic data, demo accounts

2.2 Out of scope

- Real payment processing, real shipping or carrier integration
- Real-time chat or WebSockets (messaging is asynchronous threads only)
- Real authentication security, email, SMS or push notifications
- Real file upload or storage, real backend or database
- Multi-language, right-to-left layouts, dark mode, offline mode (see Roadmap, section 23)

2.3 Priority levels

Because the timeline is 3 build days, every feature carries a priority. P0 features must be complete for launch. P1 and P2 are built in order as time allows and are listed so the full product vision is documented.

LEVEL	MEANING	RULE
P0 - Must	Launch-critical, forms the core demo path	Complete, tested, responsive
P1 - Should	Strongly improves completeness and realism	Built after all P0 items are done
P2 - Could	Polish and extended features	Built only if time remains; otherwise documented as roadmap

---

03

Technology Requirements

LAYER	CHOICE	PURPOSE
Framework / build	React (latest stable) + Vite	UI and fast development build
Language	TypeScript (strict mode)	Type safety, shared domain types, easier backend swap
Styling	Tailwind CSS with design tokens	Responsive layout and the glass design system
Routing	React Router (data router, lazy routes)	Nested routes, guards, URL state
Server state	TanStack Query	Fetching, caching, cancellation, retries, mutations
Client state	Zustand (persist middleware)	Cart, session, wishlist, compare list, UI preferences
Forms	React Hook Form + Zod	Performant forms, schema validation, field arrays
UI primitives	shadcn/ui (Radix primitives) customised to the design system	Accessible dialogs, menus, selects, tabs, toasts
Motion	Motion (React) + Lenis smooth scrolling	Page and layout transitions, scroll smoothness
Charts	Recharts (lazy-loaded)	Vendor and admin dashboards

LAYER	CHOICE	PURPOSE
Mock API	MSW (Mock Service Worker) + seed JSON + localStorage overlay	Realistic network behaviour without a backend
Testing	Vitest + React Testing Library (+ Playwright smoke tests if time)	Logic, component and flow verification
Quality	ESLint, Prettier, TypeScript strict	Consistency
Hosting / VCS	Vercel, Git and GitHub	Live deployment and history

No API keys are required. Seed product data is a one-time snapshot (derived from a public dummy-product dataset) stored in the repository, which removes third-party uptime risk and keeps results deterministic.

04

User Roles and Access Control

ROLE	CAPABILITIES	WORKSPACE
Guest	Browse, search, view products and vendor stores, build a local cart, view compare list. Must log in to check out.	Storefront
Customer	Everything a guest can do plus checkout, orders, downloads, wishlist, reviews, disputes, addresses, notifications, apply to become a vendor.	Storefront + account
Vendor	A customer whose seller application is approved. Manages own listings, own orders, store settings, earnings. Cannot see other vendors' data.	/vendor
Admin	Full platform control: approvals, moderation, users, categories, commission, reports, disputes, first-party catalog, staff role assignment.	/admin
Support Agent	Triages disputes and tickets, replies, adds internal notes, escalates to admin, read-only order lookup. Cannot approve vendors or change commission.	/support
Delivery Rider	Sees assigned deliveries, updates delivery status, confirms delivery with a simulated code, records failed attempts.	/rider

4.1 Account model

- One account can buy at any time. Selling requires approval: an account moves through vendor status none, pending, approved, rejected or suspended.
- Admin, Support Agent and Delivery Rider are staff roles assigned by an admin (seeded for the demo).
- A user holds a list of roles, so an approved vendor is still a customer and can shop.

```
Customer --apply--> PENDING --admin approves--> APPROVED (vendor role granted)
                      |                            |
                      +--admin rejects--> REJECTED   +--admin suspends--> SUSPENDED
                          (reason shown,                 (listings hidden,
                           can reapply)                   orders still fulfilled)
```

4.2 Role-based access rules

- Route guards redirect unauthenticated users to login and unauthorized users to a 403 page.
- A central permission map drives both route guards and UI visibility (a Can component), so rules live in one place.
- The mock API handlers also return 401 / 403 for forbidden actions, mirroring a real backend. Frontend guards are a user-experience layer, not security, and this is documented.
- A demo-mode role switcher provides one-click login for each seeded role (P0, for demonstration and QA).

4.3 Seeded demo accounts (simulated)

ROLE	EMAIL	NOTES
Customer	customer@demo.test	Has order history and wishlist
New customer	newuser@demo.test	Can run the vendor application flow
Vendor	vendor@demo.test	Approved vendor with listings and orders
Admin	admin@demo.test	Full access
Support Agent	support@demo.test	Has open disputes
Delivery Rider	rider@demo.test	Has assigned deliveries

All demo accounts use a simulated password. No real credentials or personal data exist anywhere in the project.

05

Catalog and Product Model

5.1 Ownership: first-party and vendor listings

- Every product has a seller. The seller is either the platform (displayed as **Sold by Vendora**) or a vendor store (displayed with the store name linking to the vendor storefront).
- Platform products are created by admins and skip moderation. Vendor products enter a moderation workflow.
- Vendors control only their own listings: create, edit, price, stock, archive, delete.

5.2 Product types

ASPECT	PHYSICAL	DIGITAL
Variants	Size, colour and similar with per-variant stock and price	License tiers (for example personal / commercial)
Stock	Tracked per variant, low-stock warnings	Unlimited
Shipping	Address, delivery option, rider delivery	None. Instant access after payment
Payment	Simulated card or Cash on Delivery	Simulated card only (no COD)
Fulfilment	Placed to delivered lifecycle	Placed to paid to available in the Downloads library with a simulated license key

5.3 Listing moderation status

Draft, pending review, approved, rejected (with reason), archived. Only approved listings from non-suspended sellers appear publicly.

5.4 Core product fields

id, title, slug, description, category, brand, images, price, compare-at price, type (physical or digital), sellerId (platform or vendor id), variants, stock, shipping settings or digital asset info, rating and review count, status, created date.

06

Pages and Routes

AREA	ROUTES	PRIORITY
Public	/ | /products | /products/:productId | /vendors/:vendorId | /cart | /compare | /login | /register	P0 (compare P1)
Customer	/checkout | /orders | /orders/:orderId | /orders/:orderId/invoice | /downloads | /wishlist | /account/profile | /account/addresses | /notifications | /disputes | /disputes/:id | /become-a-seller	P0 core, P1 rest
Vendor	/vendor | /vendor/products | /vendor/products/new | /vendor/products/:id/edit | /vendor/orders | /vendor/orders/:id | /vendor/earnings | /vendor/store-settings | /vendor/promotions	P0 (promotions P2)
Admin	/admin | /admin/vendors | /admin/products | /admin/users | /admin/categories | /admin/commission | /admin/reports | /admin/disputes	P0 vendors, products, commission, disputes. P1 rest
Support	/support | /support/disputes/:id	P1
Rider	/rider | /rider/deliveries/:id	P1
System	/403 | 404 catch-all	P0

07

Functional Requirements

7.1 Discovery and product pages (Guest and Customer)

FEATURE	REQUIREMENT	PRI
Home	Hero, categories, deals, trending, new arrivals, featured vendors, recently viewed. Each block has loading, error and empty states.	P0
Search	Debounced search with suggestions (products, categories, vendors), keyboard navigation, minimum characters, recent searches, request cancellation, stale-response protection.	P0
Listing	Filters: category, brand, price range, rating, product type, seller (platform or vendor), availability. Sorting: relevance, price, rating, newest. Pagination. All state in the URL.	P0
Product detail	Image gallery, variant selection, stock state, Sold by label, price and discount, shipping or instant-delivery info, add to cart, buy now, sticky mobile purchase bar. Independent sections for reviews, Q and A, vendor info and recommendations, each with its own states.	P0
Reviews	Rating breakdown, list with load-more, write a review (verified purchase flag), helpful votes, sort and filter.	P1
Product Q and A	Customers ask, vendor or platform answers.	P2
Wishlist	Save and unsave with optimistic update and rollback; persists.	P1
Compare	Up to 4 products side by side, difference highlighting.	P1
Recently viewed	Persisted list, capped.	P1
Recommendations	Related by category, brand and co-purchase seed data (rule-based, no AI).	P1
Vendor storefront	Banner, logo, description, rating, searchable product grid for that vendor.	P0

7.2 Cart

- Items grouped by seller. Each group shows its own subtotal, estimated shipping and fulfilment note; grand total at the bottom. (P0)
- Quantity change, remove, save for later, stock validation, notice when a price or stock changed since adding. (P0 core, save for later P1)
- Guest cart persists and merges into the account cart on login. (P0)
- Cart totals are derived, never stored. Coupon codes with simple rules (percentage or fixed, minimum amount, seller or platform scope). (P1)

7.3 Checkout (single page, accordion steps)

Checkout is one page with four collapsible steps so the user always sees progress and can edit earlier steps without route changes: (1) Shipping address, (2) Delivery option per seller, (3) Payment, (4) Review and place order. Digital-only carts skip steps 1 and 2.

- Address book with add, edit, default address, validated with Zod. (P0)
- Delivery options per seller group (standard or express, simulated fee and estimate). (P0)
- Payment: simulated card form with Luhn validation, expiry and CVV checks, and test-card guidance; or Cash on Delivery (physical only, optional small fee). (P0)
- Security rule even in simulation: card numbers and CVV are never persisted. Only brand and last four digits are stored on the order. (P0)

- Place order creates one order group with one sub-order per seller; confirmation page with summary. Double-submit protection and a final price and stock re-validation. (P0)
- Simulated payment failure path (for example a specific test card) to demonstrate error handling. (P1)

7.4 Orders, delivery and returns

FLOW	STATUSES
Physical sub-order	placed › confirmed (vendor) › packed › ready for pickup › shipped (rider assigned) › out for delivery › delivered. Alternates: cancelled (before shipped), delivery failed (retry), return requested › return approved › refunded.
Digital sub-order	placed › paid › available (download and license key shown). Alternate: refunded.
Order group	Status is derived from its sub-orders, never stored separately.

- Order list with filters, order detail with tracking timeline per sub-order, printable invoice, reorder. (P0 list, detail and timeline; P1 invoice and reorder)
- Customer can cancel before shipment and request a return within a configured window after delivery. (P1)
- Status transitions are validated by a central state machine so impossible jumps are rejected. (P0)
- Commission is calculated per sub-order at placement time and frozen on the order. (P0)

7.5 Account and notifications

- Profile editing, address management, simulated login and registration. (P0)
- In-app notification center driven by order, dispute and approval events, with read / unread state. (P1)
- Single display currency formatted through Intl.NumberFormat so localisation can be added later. (P0)

7.6 Vendor workspace

FEATURE	REQUIREMENT	PRI
Onboarding	Become-a-seller application (business name, description, category focus, contact, policy acceptance); status page showing pending, rejected with reason, or approved.	P0
Dashboard	KPIs (revenue, orders, pending orders, low stock, rating), sales-over-time chart, top products, recent orders.	P0 KPIs, P1 charts
Product management	Searchable, filterable, paginated table. Create and edit with images (simulated), variants via dynamic field arrays, stock, physical or digital settings, save as draft, submit for review. Bulk actions: publish, archive, delete.	P0 CRUD, P1 bulk
Order management	List and detail for own sub-orders; confirm, pack, mark ready for pickup; see return requests and approve or reject.	P0
Earnings	Gross sales, commission deducted, net earnings, per-order statement, payout history (simulated).	P1
Storefront settings	Logo, banner, description, policies.	P1
Promotions	Product discounts and vendor coupons.	P2
Customer messages	Per-order asynchronous message thread (not real-time).	P2

7.7 Admin workspace

FEATURE	REQUIREMENT	PRI
Vendor approval	Queue of applications, review details, approve or reject with reason, suspend and reinstate.	P0
Product moderation	Queue of pending listings with preview, approve or reject with reason, take down approved listings.	P0
Commission settings	Global default percentage, per-category override, per-vendor override, effective-rate preview, changes affect only future orders.	P0
Dispute handling	Queue with status and assignee, view thread and order, resolve with full refund, partial refund or denial, internal notes.	P0
User management	Search, filter by role and status, view profile, suspend or reactivate, assign staff roles.	P1
Category management	Category tree create, rename, reorder, hide; set commission override; prevent deleting categories in use.	P1
Reports	GMV, orders, commission earned, top vendors, top categories, date-range filter, export CSV.	P1
First-party catalog	Create and manage products sold by the platform.	P0

7.8 Support Agent and Delivery Rider (P1)

- Support: dispute and ticket queue (unassigned, mine, escalated), thread view, canned replies, internal notes, escalate to admin, read-only order lookup.

- Rider: list of assigned deliveries, delivery detail with address, status updates (picked up, out for delivery, delivered, failed attempt with reason), simulated delivery code confirmation. Assignment is automatic (seeded rule) when a vendor marks a sub-order ready for pickup, with admin override.

08

Data Layer and Mock API Design

8.1 Architecture of the data layer

```
UI components
     |  (hooks only: useProducts, useOrders, ...)
TanStack Query hooks  <---- query keys, cache, cancellation, retries
     |
Service layer (src/features/*/api)   fetch client, typed, Zod-validated responses
     |   HTTP  (fetch to /api/...)
MSW handlers  <---- in-browser service worker intercepts requests
     |
Mock database = immutable seed JSON  +  mutable overlay in localStorage

Later:  remove MSW, point VITE_API_URL at a real backend. UI code does not change.
```

8.2 Key rules

- Components never import the mock database. They only call hooks, which call services, which call HTTP endpoints.
- Seed plus overlay: the large catalog is a static seed file; only changes (new orders, vendor-created products, reviews, status changes) are written to localStorage. This avoids the browser storage size limit.
- Namespaced and versioned storage keys (for example vd:v1:orders) with a migration hook so schema changes do not corrupt old data.
- Simulated latency (random 200 to 800 ms) and a failure-injection switch (global failure rate and per-endpoint) to exercise loading, error, retry and partial-failure UI.
- Handlers enforce authentication, authorization, validation and the order state machine, returning realistic status codes.
- A Reset demo data action restores the seed.
- Storage failures (quota exceeded or disabled storage) are caught and surfaced, never crashing the app.

8.3 Domain entities

User, VendorProfile, Product, Variant, Category, Review, Question, Cart, Order (group), SubOrder, Payment (masked), Address, Dispute, Message, Notification, Coupon, CommissionRule, Delivery.

8.4 Representative endpoints

AREA	ENDPOINTS
Auth	POST /api/auth/login, POST /api/auth/register, GET /api/auth/me
Catalog	GET /api/products?q&category&brand&minPrice&maxPrice&rating&type&seller&sort&page&limit, GET /api/products/:id, GET /api/search/suggestions?q, GET /api/categories, GET /api/vendors/:id
Reviews	GET /api/products/:id/reviews?page, POST /api/products/:id/reviews
Orders	POST /api/orders, GET /api/orders, GET /api/orders/:id, PATCH /api/suborders/:id/status, POST /api/suborders/:id/return
Vendor	POST /api/vendor/apply, GET /api/vendor/products, POST /api/vendor/products, PATCH /api/vendor/products/:id, GET /api/vendor/orders, GET /api/vendor/stats
Admin	GET /api/admin/vendors, PATCH /api/admin/vendors/:id (approve, reject, suspend), GET /api/admin/products/pending, PATCH /api/admin/products/:id, GET and PUT /api/admin/commission, GET /api/admin/reports
Disputes	POST /api/disputes, GET /api/disputes, PATCH /api/disputes/:id, POST /api/disputes/:id/messages
Delivery	GET /api/rider/deliveries, PATCH /api/deliveries/:id

09

State Architecture

State is classified by what it is, and each class has one home. The central rule: server data is never copied into a client store.

KIND OF STATE	EXAMPLES	WHERE IT LIVES
Server state	Products, product detail, orders, vendors, disputes, reports	TanStack Query cache
Persistent client state	Cart, session, wishlist (guest), compare list, recently viewed, UI preferences	Zustand stores with persist middleware
URL state	Search text, filters, sort, page, selected tabs	URL search params (single source of truth)
Form state	Checkout, product editor, application forms	React Hook Form
Ephemeral UI state	Open dialogs, accordion step, hover, local toggles	Local useState / useReducer next to the component
Derived state	Cart totals, order group status, commission amounts	Computed with selectors or useMemo, never stored

- Query key factories per feature, consistent invalidation after mutations.
- Zustand access only through selectors so components subscribe to the slice they use.
- Nested state (cart groups, order group with sub-orders) is normalised by id where it is updated frequently.
- Optimistic updates with rollback for wishlist toggle and cart quantity. (P1)

10

React Rendering Requirements

10.1 Conditional rendering and API states

Every data-driven section represents the states: idle, loading, success, error, empty and refetching. This is implemented once as a reusable QueryState / AsyncSection component and a skeleton library, not repeated in each page.

- Product detail has independent sections (core product, reviews, vendor info, recommendations). If reviews fail, the rest of the page works and the failed section shows a retry control.
- Skeleton loaders match final layout to prevent layout shift.
- Refetching shows a subtle indicator without replacing existing content.

10.2 Preventing unnecessary re-renders

- Correct state placement and component composition (children pattern) before any memoisation.
- React.memo for product cards in lists, with stable props. useCallback only for handlers passed to memoised children. useMemo only for expensive derivations such as filtering large lists and report aggregation.
- useTransition or useDeferredValue where filter or search input would otherwise block typing.
- Stable keys (ids, never array index in dynamic lists).
- Every optimisation is justified in docs/performance.md with Profiler evidence (before and after for at least three cases). Memoisation is not applied blindly.

10.3 Large lists

The catalog is paginated so hundreds of cards are never rendered at once. Admin and vendor tables are paginated. List virtualisation is introduced only if profiling shows a real need.

11

Async Behaviour: Debounce, Cancellation, Race Conditions

- Search input is debounced (about 300 ms) using a custom useDebouncedValue hook written for this project.
- Requests receive an AbortSignal; superseded requests are cancelled (TanStack Query signal passed to fetch).
- Query keys include the search parameters, so a late response for an old term can never overwrite newer results.
- Suggestions dropdown supports keyboard navigation and closes correctly on outside click and escape.
- Mutations disable their trigger while pending and handle failure with rollback and a clear message.
- A test demonstrates the race scenario: slow response for term A arriving after fast response for term B must not be displayed.

12

URL State, Pagination and Routing

Example: /products?q=headphones&category=electronics&type=physical&seller=platform&minPrice=20&maxPrice=200&rating=4&sort=price_asc&page=2

- Search, filters, sort and page live in the URL. Refresh restores state; Back and Forward work correctly; links are shareable.
- URL parameters are parsed and sanitised with Zod; invalid values fall back to defaults.
- Changing any filter resets the page to 1.
- Product listing uses pagination (shareable pages). Review lists use load-more.
- All role areas, checkout and chart-heavy pages are lazy-loaded with React.lazy and Suspense, so the public landing page does not download vendor or admin code.
- Error boundaries exist at three levels: application, route, and section. Fallback UI offers a Try again action.

13

UI / UX Design Requirements

13.1 Visual direction: bright glass

- Light, vivid interface: bright colour fields behind translucent frosted surfaces.
- Pill language: floating pill navigation bar, pill search field, pill buttons, pill filter chips and badges. Cards use large radii.
- Glass recipe: semi-transparent white fill, backdrop blur, thin light inner border, soft colour-tinted shadow.
- Suggested palette (tokens in Tailwind theme): Sun Orange #FF7A1A, Aqua #00B8C8, Coral #FF4D6D, Lemon #FFD43B, Ink #14213D, Mist #F4F7FB. Final tuning during build.
- Typography: a distinctive display face for headings paired with a clean text face (for example Bricolage Grotesque and DM Sans), loaded with font-display swap.
- Icons from one consistent set (lucide). Real product imagery. Realistic content, no placeholder lorem ipsum.

13.2 Keeping it crafted, not generic

- Avoid the stock centred hero over a purple-blue gradient. Use asymmetric layouts, overlapping elements and varied section rhythm (for example bento-style blocks).
- One clear focal point per page; consistent spacing scale; considered empty states and micro-copy written for the product.
- No emoji as icons, no neon glow overuse, no identical card grids repeated down the whole page.

13.3 Glass readability and performance rules

- Body text must meet contrast of 4.5:1 on glass surfaces; raise surface opacity where needed.

- Reserve blur for navigation, drawers, modals and hero panels. Product grids use near-solid cards to avoid scroll jank on phones.
- Limit stacked blur layers on screen and provide a solid fallback where backdrop-filter is unsupported.

13.4 Motion

- Lenis for smooth scrolling, paused when modals and drawers are open and disabled under prefers-reduced-motion.
- Motion library for page transitions, staggered grid entrances, layout animations (cart changes, filters) and micro-interactions.
- Animations are short, purposeful and never block interaction. Skeleton shimmer for loading.

13.5 Responsive design

- Mobile-first; tested at 360, 390, 768, 1024, 1440 and 1920 pixel widths and on real devices.
- Mobile patterns, not shrunken desktop: bottom pill navigation, bottom-sheet filters, sticky add-to-cart bar, tables transformed into card lists in vendor and admin areas, touch targets of at least 44 px.
- Navigation, cards, forms, tables, dashboards, modals and filters each have an explicit mobile design.

13.6 Accessibility (basic level)

- Semantic HTML, labelled form controls, meaningful alt text, visible focus states, skip-to-content link.
- Keyboard operability for menus, dialogs, dropdowns and the suggestion list (largely via Radix primitives).
- aria-live announcements for toasts, respect for prefers-reduced-motion, sufficient contrast.
- Deeper accessibility work (full screen-reader audits, focus management refinement) is deferred to a later phase.

14

Performance Expectations

- Route-level code splitting and lazy chart library; bundle analysed with a visualiser and documented.
- Images: lazy loading, fixed aspect-ratio boxes to prevent layout shift, appropriately sized sources.
- Request frequency controlled by debouncing, caching (staleTime) and cancellation.
- Targets on the home page (mobile Lighthouse): Performance 85 or higher, Accessibility 90 or higher, Best Practices 90 or higher.
- Dependency discipline: every library has a reason documented in docs/decisions.md.

15

Testing Strategy

LAYER	WHAT IS TESTED	TOOLS	PRI
Unit	Cart grouping and totals, commission calculation, order state machine, permission map, URL param parser, useDebouncedValue	Vitest	P0
Component	QueryState component states, product card, checkout form validation, route guards	Vitest + React Testing Library	P0
Integration	Search race condition with delayed MSW handlers, login and role redirect, cart to order creation	Vitest + RTL + MSW (same handlers as the app)	P1
End-to-end smoke	Guest to COD checkout; vendor applies, admin approves, vendor lists product	Playwright	P2

Tests are written alongside the logic they protect (not saved for the end). The QA day executes the manual role-by-role test matrix and fixes defects.

16

Git, Code Quality and Structure

16.1 Git and GitHub

- Repository created on day one; small, meaningful commits using Conventional Commit style (feat, fix, refactor, test, docs, chore).
- Examples: feat: add debounced product search, fix: ignore stale search responses, refactor: extract order state machine, feat: add vendor approval queue.
- Feature branches merged to main with clear messages; at least one commit per completed feature. No single giant final commit.
- Optional CI (lint, typecheck, test, build) and automatic Vercel preview deployments. (P1)

16.2 Code quality

- TypeScript strict with no any; shared domain types; Zod validation at API boundaries.
- Small focused components, separation of concerns, no excessive prop drilling, no duplicated logic.
- No hard-coded application data in components; all data via services or seed.
- ESLint and Prettier clean; no unused code; no unnecessary dependencies; meaningful names.

16.3 Folder structure

```
src/
  app/            providers, router, query client, error boundaries
  components/
    ui/           shadcn primitives (customised)
    common/       QueryState, Skeletons, EmptyState, Pagination
    layout/       Navbar (pill), BottomNav, Footer, RoleLayouts
  features/
    auth/ catalog/ product/ cart/ checkout/ orders/ wishlist/ reviews/
    vendor/ admin/ support/ rider/ disputes/ notifications/
      (each: components/ hooks/ api/ types.ts utils/)
  pages/          thin route components
  hooks/          useDebouncedValue, useLocalStorage, ...
  lib/            fetch client, formatters, zod schemas, rbac
  mocks/          handlers/, db/, seed/, browser.ts, server.ts
  store/          zustand stores
  styles/         tokens, globals
  types/
docs/             architecture.md, performance.md, decisions.md, qa-report.md
```

17

Deliverables

DELIVERABLE	DETAILS
Live deployment	Vercel, with SPA rewrite configuration so deep links and refresh work; MSW enabled in production build.
GitHub repository	Complete source and meaningful commit history.
README	Overview, features, tech stack, how the mock API works, installation, environment variables, demo accounts, project structure, state management approach, key technical decisions, screenshots, known limitations.
Architecture diagram	Layers, data flow, state ownership, role areas (included in README and docs).
Documentation	docs/architecture.md, decisions.md, performance.md (Profiler evidence), qa-report.md.
Demo video	3 to 5 minute walkthrough covering all roles and the technical highlights.
QA report	Role-by-role test matrix, device checks, Lighthouse results, defects fixed.

18

Four-Day Delivery Plan

DAY	FOCUS	OUTCOME
Day 1	Repo, Vite, TypeScript, Tailwind, router, tokens and glass components. MSW, seed data, mock database, service layer, query client. Auth with RBAC and demo switcher. Layout and pill navigation. Home, search with debounce and cancellation, listing with URL state, product detail, vendor storefront.	A guest can browse, search, filter and open products. Roles can log in.

DAY	FOCUS	OUTCOME
Day 2	Cart (grouped by seller), checkout (card and COD), order creation with sub-orders and commission, order list, detail and tracking, state machine. Seller application. Vendor dashboard KPIs, product CRUD, order handling. Wishlist and reviews.	Full buy-and-sell loop works end to end.
Day 3	Admin: vendor approval, moderation, commission, disputes, first-party catalog, then users, categories, reports. Support and rider workspaces. Notifications. Motion, Lenis and visual polish. Responsive pass, error boundaries, lazy loading. README and deploy.	All roles functional; deployed build live.
Day 4 (QA)	Manual test matrix by role, device widths, Lighthouse and bundle review, accessibility basics, bug fixing, remaining unit and integration tests, documentation, architecture diagram, demo video.	Release-ready submission.

> *Priority rule for the schedule: finish all P0 items first, then P1, then P2. Unfinished P1 and P2 items are documented as roadmap rather than shipped half-built.*

19

Demo Plan

The developer will demonstrate, in this order:

- Guest browsing: search with suggestions, filters and pagination with URL state, refresh and Back / Forward behaviour
- Product detail with independent sections and a simulated partial failure with retry
- Customer purchase with a multi-vendor cart: two sellers, one platform item, one digital item; card and COD
- Order tracking and the order lifecycle across vendor, rider and delivered
- Customer applies to become a vendor; admin approves; vendor lists a product; admin moderates it
- Vendor dashboard, product management and earnings after commission
- Dispute raised by a customer, triaged by support, resolved by admin
- Admin commission change and its effect on a new order only
- Role-based access: forbidden routes and the 403 experience
- Responsive behaviour on mobile, tablet and desktop; loading, error and empty states; glass design and motion

---

20

Technical Discussion Topics

During the review, the developer is prepared to discuss the following areas and the reasoning behind each implementation decision.

AREA	DISCUSSION TOPICS
React rendering	Why specific components re-render; where each optimisation was introduced and the evidence for it; effect of moving state to a parent; selectors and subscription granularity; composition versus memoisation.
State	Separation of server state, client state, URL state and form state; why Zustand and TanStack Query were chosen; how the cart, nested orders and commissions are modelled; persistence and schema versioning.
Async and API	Error handling and partial failure; debouncing implementation; request cancellation; behaviour when responses arrive out of order; retry and refetch strategy; optimistic updates and rollback.
Architecture	Feature-based structure; reusable components; the mock API boundary and the path to a real backend; scaling the application; where validation lives.
Routing	Why state is in the URL; behaviour on refresh and navigation; lazy loading and chunking; guards and error boundaries.
Marketplace logic	Seller-grouped cart and split orders; order state machine; commission calculation and freezing; vendor approval workflow; first-party versus vendor listings.
Access control	Permission map design; difference between UI guards and backend enforcement; how the mock handlers mirror real authorization.
Design and performance	Glass design system trade-offs; where blur is used and why; Lenis integration; responsive strategy; bundle and Lighthouse results.

21

Risks and Mitigations

RISK	MITIGATION
Scope is large for four days	Strict P0 / P1 / P2 ordering; vertical slices; unfinished items become documented roadmap
Browser storage limits and corruption	Seed plus overlay model, versioned keys, error handling, reset action
Glass effects hurt mobile performance	Blur only on a few surfaces; near-solid cards in grids; solid fallbacks; test on real devices
Mock service worker setup issues in production	Enable in production build, verify on a Vercel preview on day 1, keep a simple fallback adapter
Smooth scrolling conflicts with modals and inner scroll areas	Pause Lenis when overlays are open; mark inner scroll containers; respect reduced motion
Deep-link refresh returns 404 on hosting	SPA rewrite configuration and verification in QA

22

Definition of Done and Acceptance Criteria

- All P0 features work for every role and pass the QA matrix on mobile, tablet and desktop.
- Search is debounced, cancellable and race-safe; filters and pagination restore from the URL.
- Every data-driven section has loading, error, empty and (where relevant) refetching states.
- Role guards work; forbidden routes show 403; handlers enforce permissions.
- Orders split by seller; commission calculated, frozen and visible in vendor earnings and admin reports.
- No console errors in normal flows; TypeScript, lint and tests pass; production build succeeds.
- README, architecture diagram, documentation and demo video delivered; live URL works including refresh on deep links.
- Each optimisation has a written reason; the developer can explain state placement and re-render behaviour.

23

Roadmap: Connecting a Real Backend

The service layer and typed, validated API contracts make the next phase additive rather than a rewrite:

- Replace MSW with a real API (Node / NestJS, Supabase, or a WooCommerce / headless commerce backend) by setting the API base URL and removing the worker.
- Real authentication (tokens, refresh, secure cookies), server-side authorization and validation.
- Real payments and webhooks, shipping integrations, file storage for images and digital assets.
- Real-time chat and notifications, search engine for the catalog, email notifications.
- Offline support and installable PWA, dark mode, localisation, deeper accessibility, expanded automated testing.

24

Bonus Features

- Drag-and-drop reordering of vendor product images and category tree.
- Optimistic updates beyond wishlist and cart; cross-tab synchronisation of cart and session via the storage event.
- Dark theme, offline support, PWA installability, expanded accessibility, end-to-end test suite.